BODH.AI — Presentation Script (Slides 1–15)

Target: ~17 minutes

---

1. Cover — [0:30]

Good [morning/afternoon], everyone. Thank you for being here for my M.Tech thesis defense.

Today I'll be presenting BODH.AI — a national-scale ecosystem for privacy-preserving federated learning and independent model evaluation in healthcare. This is joint work with my co-author Harsh Baid, under the supervision of Prof. Nisheeth Srivastava. My part focuses on building the resilient infrastructure layer, and Harsh will follow with the benchmarking and deployment layer.

---

2. Agenda — [0:45]

Here's how the talk is structured. I'll start with the problem and motivation — why healthcare AI has a trust crisis. Then I'll cover the resilient FL infrastructure I built. Harsh will take over for BODH.AI's benchmarking and deployment work. After that, we'll walk through a national-scale hackathon we used to stress-test the whole system, then close with our combined contributions, limitations, and future work.

---

3. The Problem — [1:30]

Let's start with why this problem matters.

Healthcare AI faces two structural crises. First, **data silos**. Patient records are sensitive and heavily regulated — you legally and ethically cannot centralize them for training a model. Every hospital's data has to stay where it is.

Second, a **trust deficit**. Right now, most clinical models are evaluated by the same people who built them, on their own internal test sets. That invites overfitting, subtle data

leakage, and inflated performance claims — there's no independent check.

Federated learning is the standard answer to the first problem — train without moving the data. But in practice, no existing framework actually survives real-world hospital deployment, and there's no trustworthy, independent way to evaluate the resulting models. That's the gap this thesis addresses.

---

4. How the Two Theses Fit — [1:00]

Our work splits into two tightly connected parts, but it's one ecosystem.

My part — Part A — is the resilient infrastructure: the communication, security, and scalability layer that lets federated learning actually survive real hospital networks, not just a clean lab simulation.

Harsh's part — Part B — is the trust layer: independent evaluation, accessible deployment, and verifiable evidence that clinicians can actually rely on.

The shared goal across both: make FL production-ready, and make its outputs trustworthy.

---

5. FL Primer — [1:30]

Quickly, for anyone who wants a refresher — federated learning in one slide.

The core idea: data stays local, on the hospital's own servers. Only model weight updates ever leave the building. A central server collects those updates from many participants and aggregates them into a single global model.

The academic assumption behind most FL research is that communication is instant, reliable, and free. The reality on the ground is very different — intermittent connectivity, firewalls, and wildly heterogeneous hardware across hospitals. That gap between the assumption and the reality is exactly what my part of the thesis is about.

---

6. Section Divider — Part A — [0:30]

So let's get into it — Part A, resilient infrastructure. This is about turning a brittle research prototype into a production-grade federated learning system that actually survives real hospital networks.

---

7. Old System — [1:15]

Here's where we started, and why it failed.

The original pipeline used a persistent, synchronous, server-driven socket connection. The server held an open connection to every client for the entire training round. The problem: hospital firewalls and NAT configurations routinely broke these connections, and a single client dropout could stall or doom the entire round.

At national scale — thousands of concurrent nodes — this design was architecturally impossible. It simply could not scale.

---

8. New System: Pub/Sub — [1:30]

So the core contribution here is a redesign around an event-driven Publish-Subscribe architecture.

Instead of holding connections open, the server publishes events — like a `ROUND_START` event — to topic channels. Clients subscribe to these channels, fetch the current model weights over a stateless REST call, disconnect, train locally on their own time, and then push their updates back whenever they're ready.

The effect: the server becomes an asynchronous state machine, not a blocking loop that's waiting on every client. Server state is now fully decoupled from any single client's connectivity — one hospital going offline doesn't take down the round for everyone else.

---

9. Pub/Sub Flow Diagram — [1:00]

(Walk through the diagram on screen.)

This diagram shows that flow end-to-end — the server publishing the round event, clients pulling weights independently, training locally, and pushing updates back asynchronously. Notice there's no persistent connection anywhere in this loop — every interaction is a short, stateless request. That's what makes it resilient to exactly the kind of network conditions real hospitals have.

---

10. Key Transformations — [1:15]

To summarize the redesign, here are five key before-and-after transformations.

Communication moved from a persistent socket to a stateless REST API. Training moved from a server-controlled loop to event-triggered. Load moved from high and centralized to distributed, with no heartbeat overhead. Connectivity requirements moved from needing continuous uptime to being intermittent-tolerant. And scalability moved from vertical-only — bigger servers — to horizontal — just add more nodes.

Together, these five changes are what take this from a research prototype to something that can actually run at national scale.

---

11. Secure Aggregation — [1:30]

Resilience alone isn't enough — we also need privacy guarantees, so let's talk security.

The goal of secure aggregation, or SecAgg, is for the server to compute the *sum* of all client updates without ever seeing any single client's individual weights.

We do this through pairwise secret sharing — clients add random masks to their updates before sending them. These masks are constructed so they cancel out exactly when the server sums everything together. So if anyone intercepts a payload in transit, it just looks like meaningless noise — even the server itself can't isolate one client's contribution.

The tricky part is handling dropouts — if a client disappears mid-round, its mask can't just cancel out. We use Shamir's Secret Sharing for dropout recovery, and we had to re-

engineer it to work in this new stateless, asynchronous world rather than the classic synchronous setting it was designed for.

---

12. Data Pipeline — [1:15]

Moving to some of the pragmatic engineering work — first, the edge data pipeline.

The original system used Hadoop and Spark for local data loading, but the JVM overhead was actually starving the GPU during training — most of the time was spent in data pipeline overhead, not learning. Empirically, we found hospital datasets rarely exceed gigabyte scale, so a distributed file system was simply unjustified for this use case.

We rearchitected this down to local processing — Pandas and native PyTorch DataLoaders. The result: a single lightweight executable, orders-of-magnitude faster data loading, and — just as important — higher trust from hospital IT admins, who no longer need to run a Hadoop cluster to participate.

---

13. Database — [1:15]

Second piece of engineering — the orchestration database.

We started with SQLite, which suffered catastrophic locking failures the moment we had concurrent writes at national load — multiple hospitals trying to update round status at the same time would just deadlock.

We migrated to PostgreSQL, which gives us MVCC — multi-version concurrency control — and proper connection pooling. Now the system handles hundreds of simultaneous writes without data loss or timeout cascades.

---

14. Convergence — [1:15]

Before moving to Part B, one validation question: does this FL implementation actually learn correctly?

We ran a controlled MNIST experiment that simulates hospital data-silo conditions — data split across simulated "hospitals" that never share raw data. We compared three settings: centralized training as the upper bound, isolated training with no collaboration, and our federated setup.

(Point to the convergence plot.) This confirms our FL implementation converges correctly and tracks the centralized upper bound closely — meaning the resilience and security work in the earlier slides isn't coming at the cost of actual learning performance. This is distinct from the scale and concurrency stress test, which we'll get to shortly.

---

15. Section Divider — Part B — [0:45]

That covers the infrastructure layer — making federated learning resilient and secure enough to survive real hospital networks.

Now I'll hand it over to Harsh, who'll walk you through Part B: benchmarking and deployment — how BODH.AI turns this infrastructure into independent, privacy-preserving evaluation and one-click hospital participation, turning model claims into verifiable evidence.